

FlagOS多芯片统一vLLM插件——vllm-plugin-FL

vllm-plugin-FL: a plugin for the vLLM inference/serving framework, built on FlagOS's unified multi-chip backend

智源推理研发工程师 陈卓



Content

01 FlagOS

02 vllm-plugin-FL

03 Future work



A Unified, Open-Source AI System Software Stack

FlagOS

AI大模型

语言模型

DeepSeek, Qwen, Seed-oss, GPT-oss, Step, Grok, Llama, GLM, Hunyuan等

多模态模型

EMU, MiniCPM-v, Qwen2.5/3-VL, ERNIE4.5, Llava系列

具身智能模型

RoboBrain2, RoboBrainX0.5, Pai-0, Pai-0.5

FlagOS 生态使能项目

Megatron-FL

vLLM-plugin-FL

TransformerEngine-FL

深度学习框架

(PyTorch, PaddlePaddle, etc)

FlagOS 开源核心库

FlagGems
通用大模型算子

FlagScale
训练推理并行框架

FlagTree
统一编译器

FlagCX
统一通信库

FlagOS 开源工具

KernelGen v1.0 算子自动生成工具

FlagRelease 自动迁移和发版工具

FlagPerf 多芯片评测工具

支持多种芯片架构

Nvidia, NPU, GPGPU, DSA, RISC-V
AI, ARM等
16家厂商25款AI芯片

支持多种系统硬件形态

端侧计算模组



AI PC



AI服务器

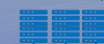


超节点系统



支持多种应用场景

数据中心
(train & Inference)



机器人
(cloud-edge cooperation)



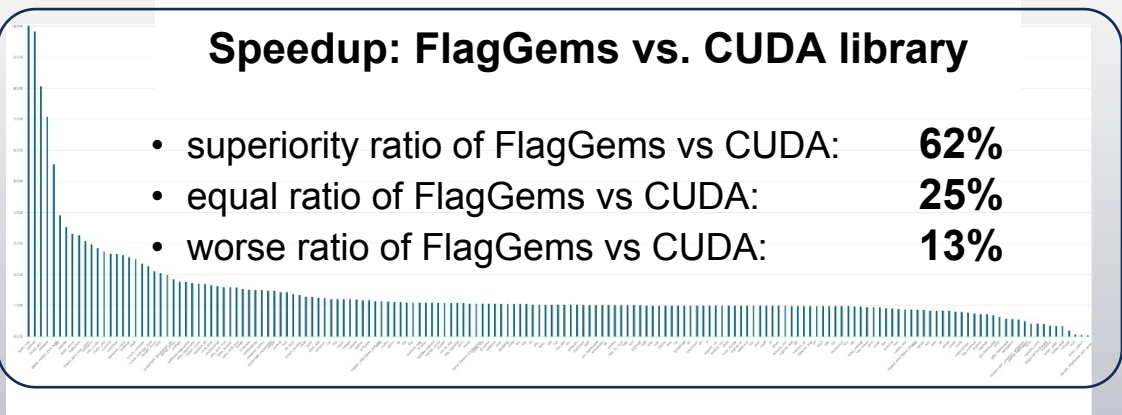
边缘
(inference)



A high-performance, generic operator library: FlagGems

FlagOS

➤ 已经发布了230个常用大模型算子，87%达到或超过CUDA算子性能



➤ FlagGems 成为PyTorch基金会生态合作项目，目前平均每周用户下载2000次



FlagOS1.6版本（2026年1月9日发布）新增138个高性能Triton算子，FlagGems算子总数将超过360个，其中296个算子性能达到或超过CUDA原生算子。

➤ 7家国产厂商适配FlagGems算子，基于FlagTree统一编译器，全部测试超过220个算子，以确保技术的泛化性

基于FlagTree统一编译器，7家国产厂商适配FlagGems算子相比厂商原生算子加速比中位数.

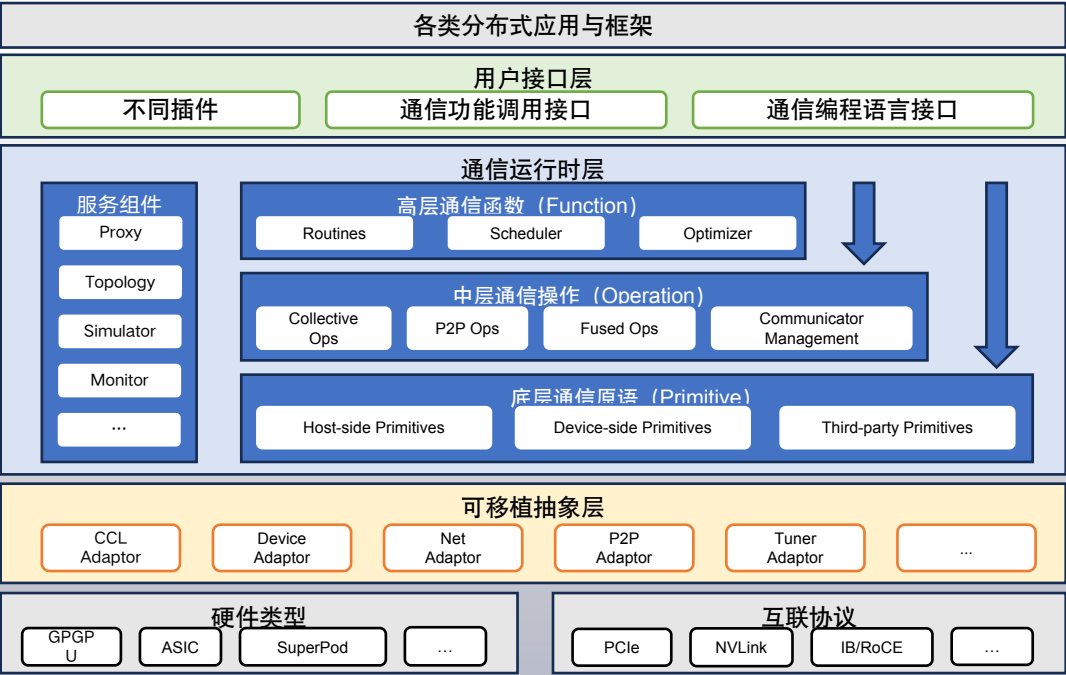
GPU A	GPU B	GPU C	GPU D	DSA A	DSA B	DSA C
112%	107%	104%	95%	96%	80%	79%

➤ PreTune 离线搜寻最优内核配置

问题：为了满足每次request的变长输入的优化，Triton使用online AutoTune，造成推理耗时陡增

解法：设计高效的离线搜寻机制，建立Shape与内核参数对应关系，online 直接查表获取内核参数，降低搜索耗时

收益案例：应用于 Qwen2.5-7B-Instruct，端到端推理性能提升**40%**。



FlagCX 统一通信库已经支持了10家芯片厂商

Vendor	Nvidia 英伟达		AMD 超威		Iluvatar 天数智芯		Cambricon 寒武纪		MetaX 沐曦		Kunlunxin 昆仑芯		Hygon 海光		Huawei 华为		Moore 摩尔线程		Enflame 燧原科技	
Mode	Hom o	Hete ro	Hom o	Hete ro	Homo	Heter o	Homo	Hetero	Ho mo	Heter o	Homo	Hetero	Ho mo	Heter o	Homo	Heter o	Homo	Hetero	Homo	Hetero
send																				
recv																				
broadcast																				
gather																				
scatter																				
reduce																				
allreduce																				
allgather																				
reducescatter																				
alltoall																				
alltoallv																				
group ops																				

- **全场景覆盖：**全面支持集合通信，涵盖同构和异构全场景，并实现多芯片自动拓扑探测功能
- **更多芯片支持：**支持英伟达、寒武纪、昆仑芯、海光、华为昇腾、摩尔线程等10种芯片
- **更多协议支撑：**支持IBRC、IBUC、RoCE、Socket、UCX等5种网络协议
- **双深度学习框架支持：**既支持PyTorch，也被原生集成到百度飞桨3.0正式发版中
- **国家国际标准同时立项：**国家标准《人工智能 统一通信库接口规范》正式立项（20255428-T-469）；ITU国际标《Requirements and Framework of Cross-Platform Unified Communication Libraries for Distributed Multimedia AI Systems》已正式通过国际标准ITU-T SG21的标准立项。

关于对《人工智能 统一通信库接口规范》等167项拟立项国家标准项目公开征求意见的通知

发布时间：2025-07-28 14:15 信息来源：标准技术司

各有关单位：

经研究，现对《人工智能 统一通信库接口规范》等167项拟立项国家标准项目公开征求意见，征求意见截止时间为2025年8月27日。请登录标准技术司网站征求意见公示网页<http://std.samr.gov.cn/gb/gbSuggestionPlan?bid=10002589>，查询项目信息和反馈意见。

2025年7月28日

➤ 兼容Triton的开源统一AI编译器

FlagTree是FlagOS生态系统的核心组件，基于Triton语言和编译器技术进行增强，打造面向多种AI硬件架构的开源、统一编译器。通过开放合作的方式，为开发者提供更多选择，推动AI系统创新技术的普及和多元发展。

➤ 核心理念

"Write once with Triton, compile anywhere"

开发者只需编写一次Triton语言代码，即可通过FlagTree编译到多种AI芯片平台，大幅降低跨平台适配成本。

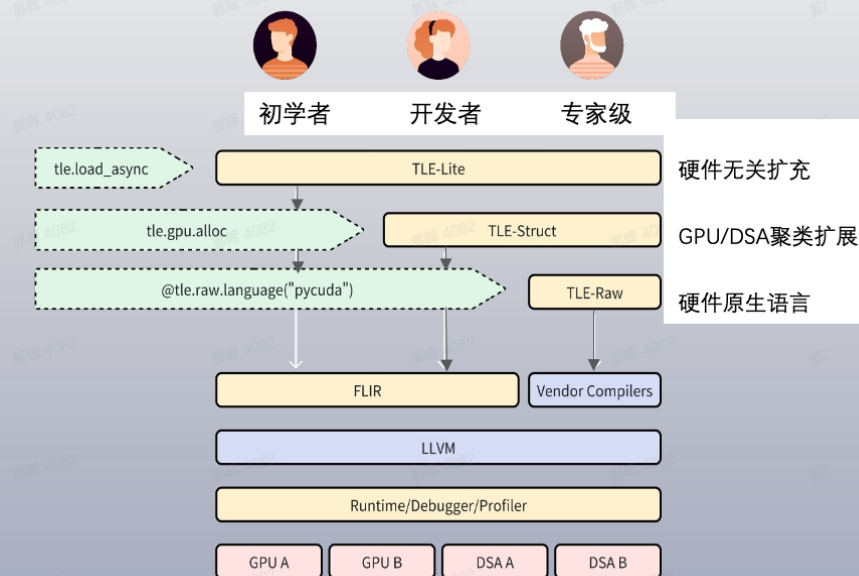
➤ 分层架构设计

- 前端语言层: 支持Triton、Helion、TLE等多种DSL，通过AST解析下降到不同层级IR
- 中端优化层: FLIR统一中间表示，兼容Linalg Dialect，针对DSA架构感知优化
- 后端代码生成: 支持NVIDIA、AMD、国产GPGPU、DSA、RISC-V AI、ARM AI等多架构

FlagTree v0.4 —— 扩展Triton语言，提升硬件感知

✓ 在原有Triton语言基础上，提出分层的语言扩展

Triton Language Extension (TLE)

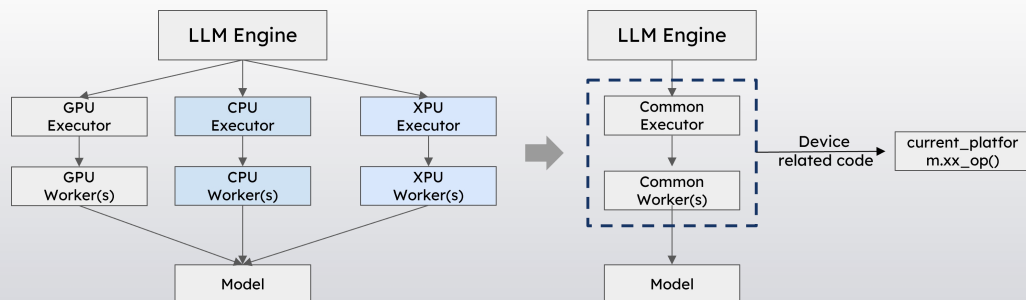


A vLLM plugin built on the FlagOS unified multi-chip backend

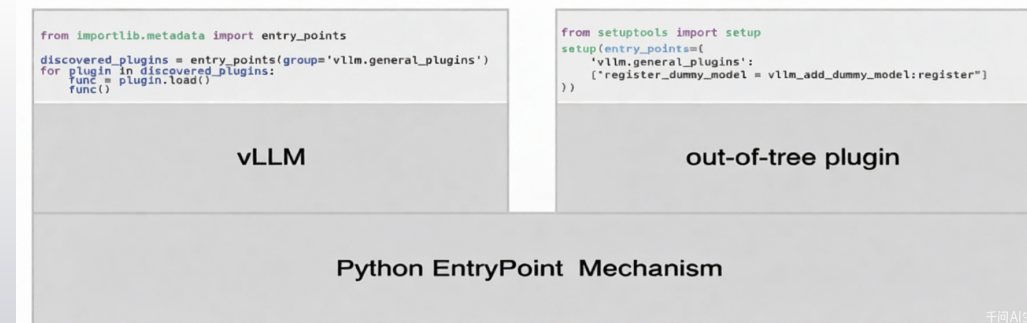
vllm-plugin-FL

FlagOS

<https://github.com/vllm-project/vllm/issues/9268>



<https://github.com/vllm-project/vllm/issues/11162>

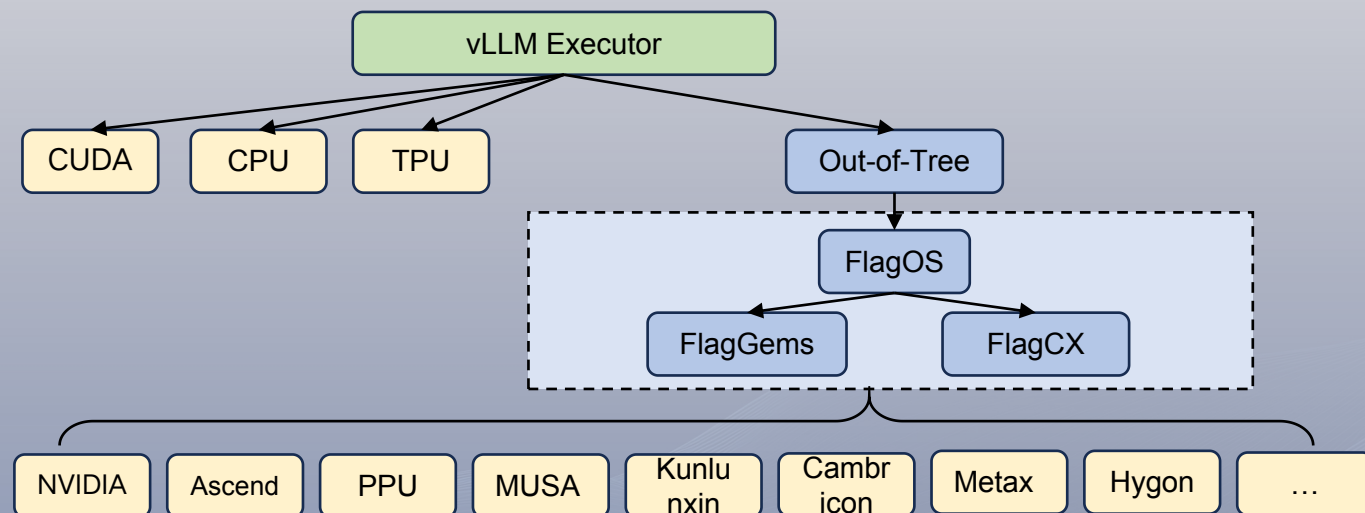


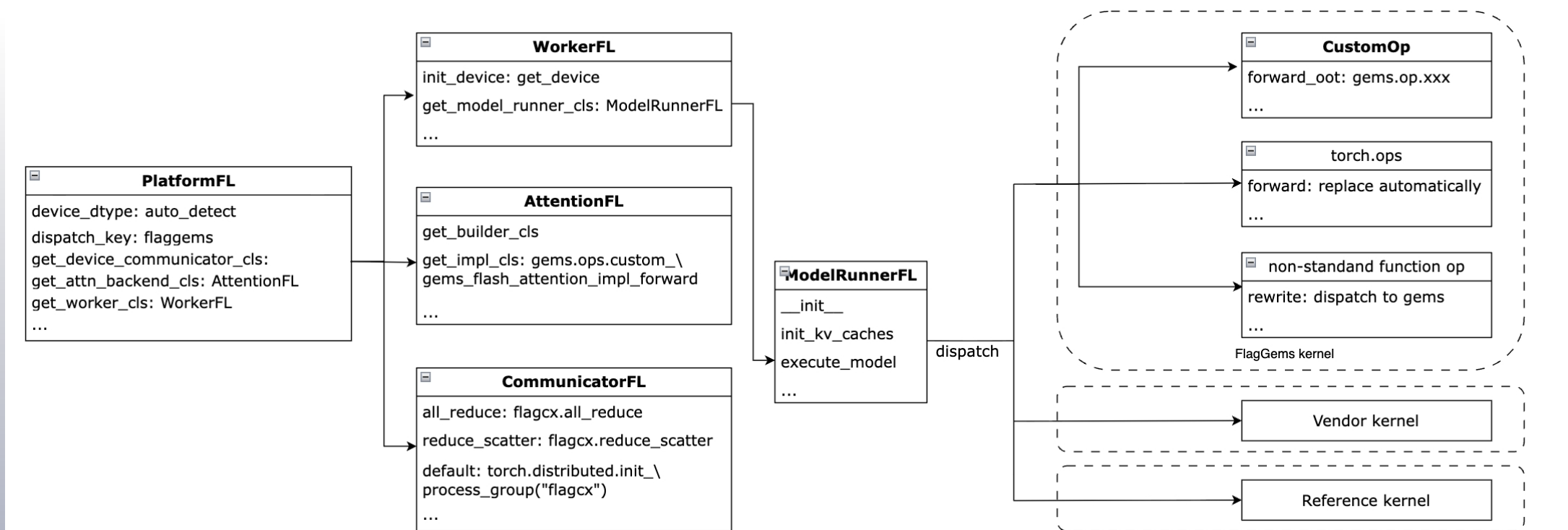
➤ vLLM多后端v1版本：代码中对硬件相关代码抽象

➤ vLLM多后端FlagOS版本：

- 以Out-Of-Tree的方式把FlagOS作为虚拟后端按照vLLM的Plugin规范接入vLLM。
- 利用FlagOS多芯片后端优势，实现框架层面硬件无关代码统一。
- 不同芯片上用户将使用相同vllm版本。
- 利用PyTorch的图层Pass优化机制实现不同芯片的自定义融合等优化。

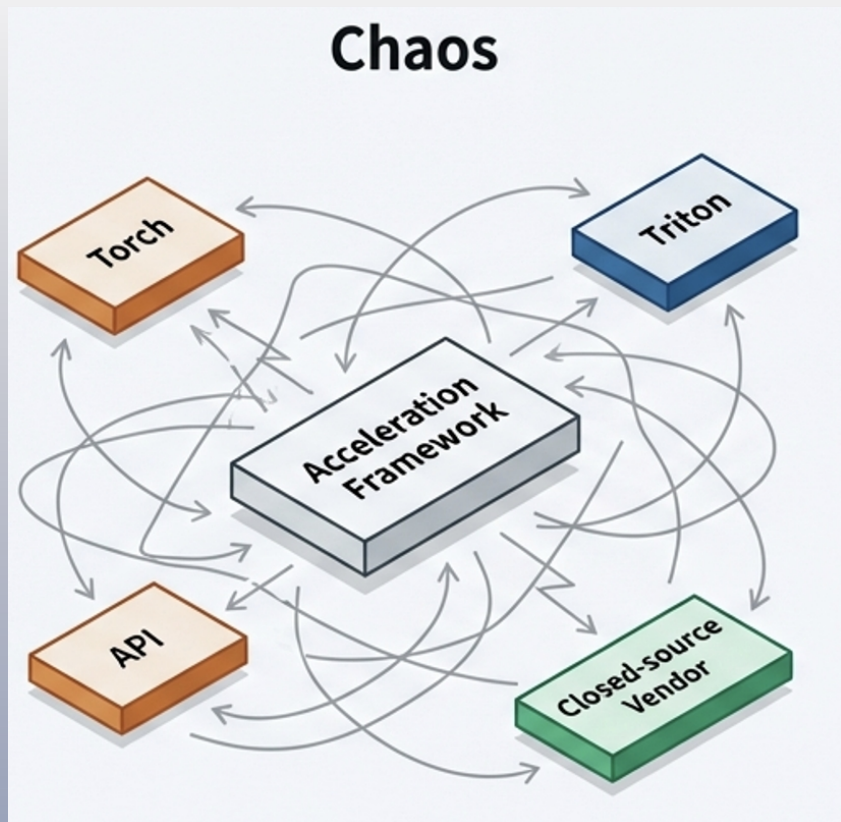
➤ vLLM多后端v2版本：基于Python EntryPoint机制





- ✓ 标准自定义算子，通过forward_out 重定向算子的执行。
- ✓ 标准PyTorch算子，基于FlagGems自动替换方式
- ✓ 实现统一算子管理，并在调用时自动进行算子dispatch。

➤ 解决底层碎片化，七大核心设计目标



兼容性: Seamless fallback mechanisms.



用户无感: End-users call unified APIs without knowing the backend.



自动发现: Zero-config loading via entry points.



分层文档: Tailored APIs for users, vendors, and framework devs.



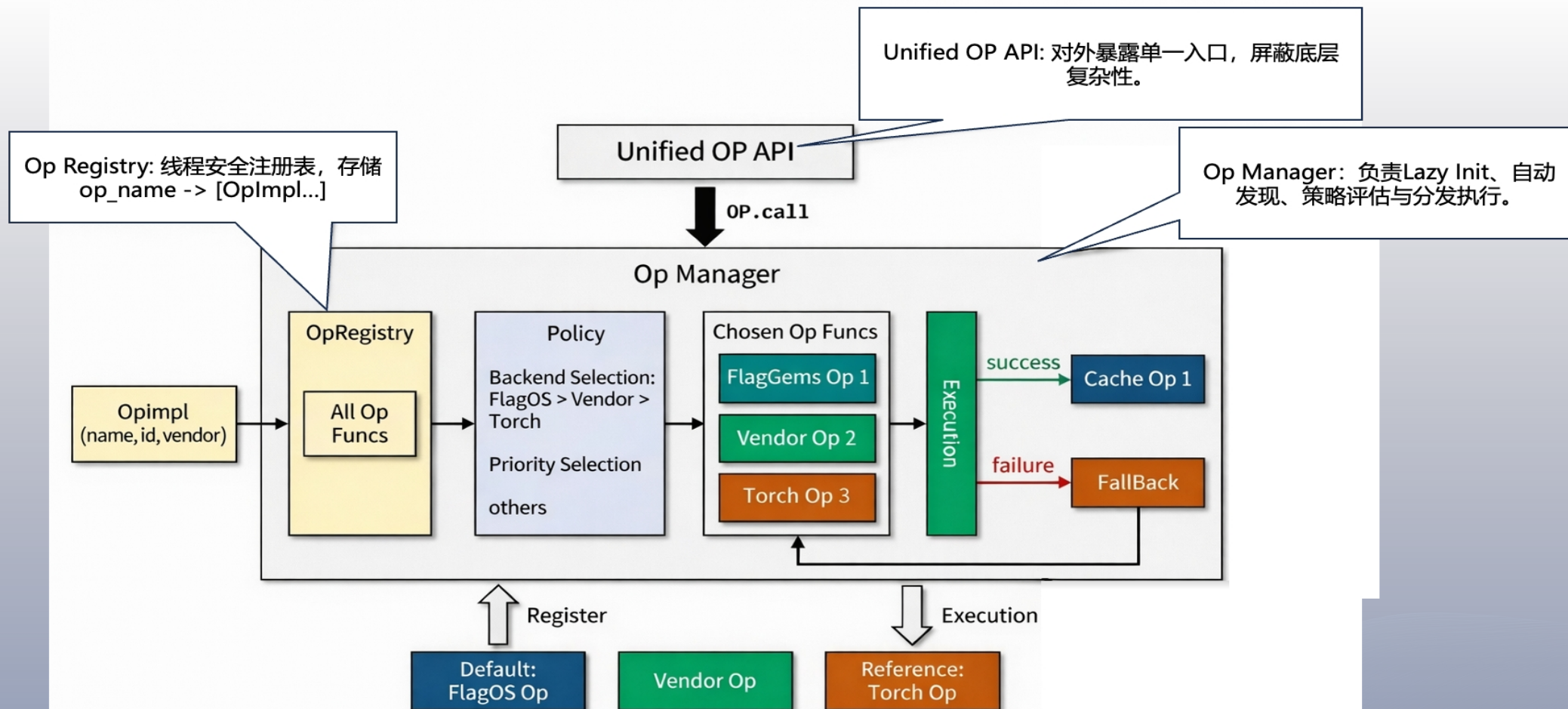
可插拔: Vendor implementations are optional and easily swapped



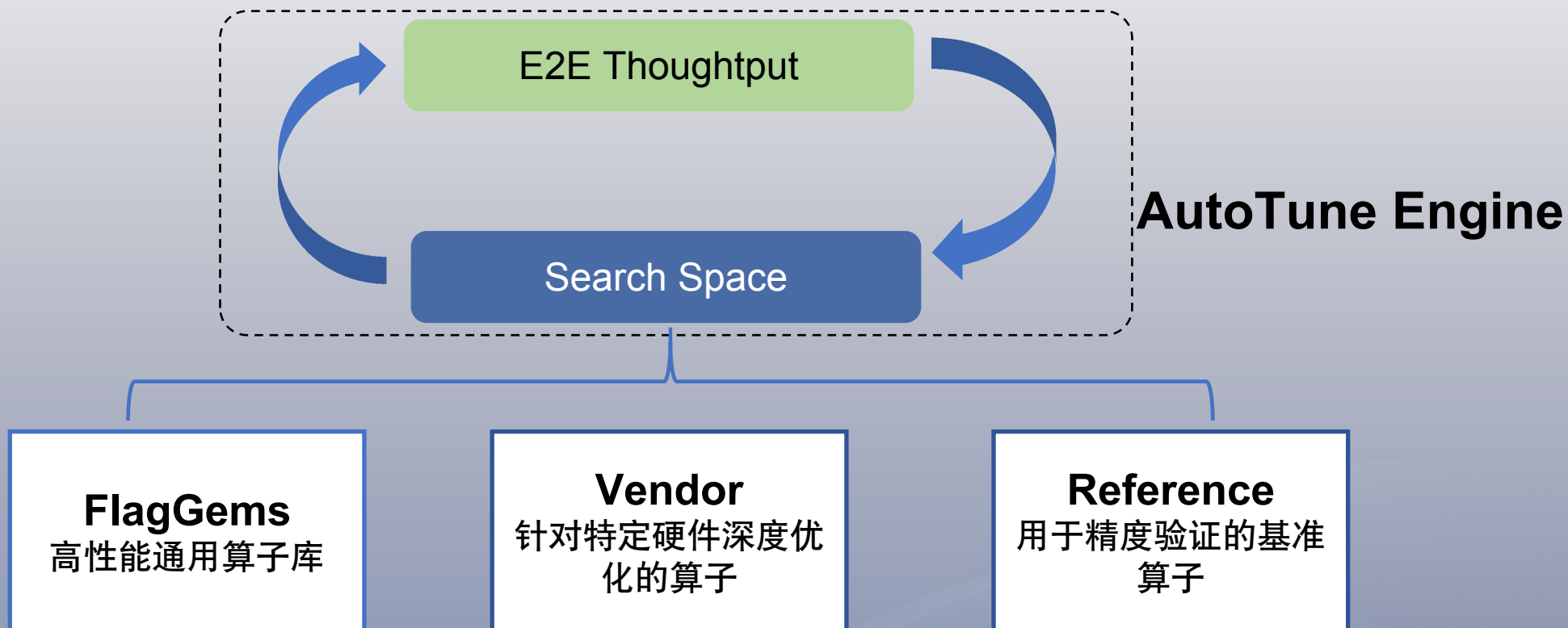
细粒度控制: Advanced users control per-op routing.

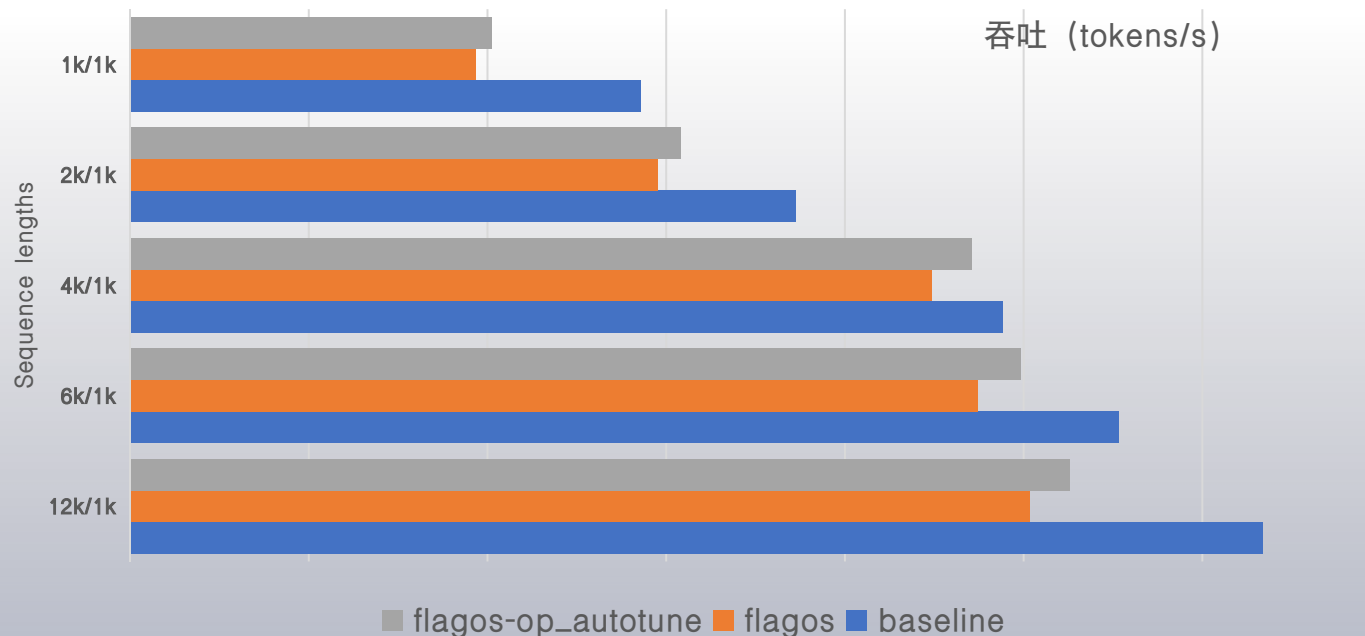


生产级: Thread-safe, multi-process aware, lazy initialization.



- 自动遍历模型算子，基于3种后端构建算子搜索空间
- 通过端到端模型基准测试，自动生成最优算子配置，实现最佳吞吐



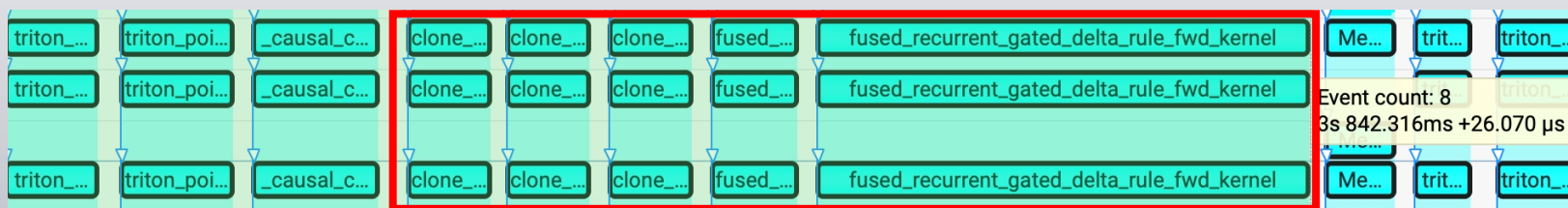


算子版本	算子名称	耗时占比
cuda	vllm::moe::moe_align_block_size_kernel	2.5%
FlagGems	moe_align_block_size_stage1 moe_align_block_size_stage2 moe_align_block_size_stage3 moe_align_block_size_stage4	21.7%

- 自动调优后moe_align_block_size算子选择cuda版本，可以实现端到端性能提升3%-4.6%。
- 自动调优对端到端性能提升有明显收益。
- 通过timeline分析模型算子情况，发现FlagGems版本的moe align算子在端到端性能中耗时占比大。

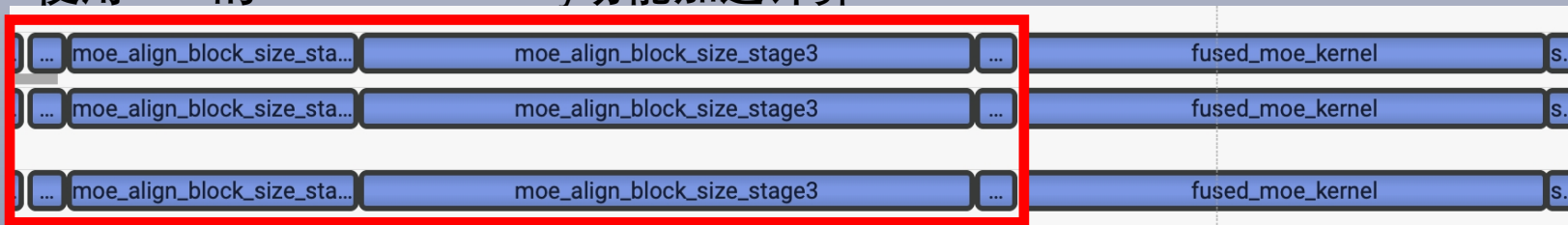
➤ 框架算子联合优化

- 重写解码阶段的GDN 算子，并增加并行度加速计算
- 框架层面移除算子调用前不必要的clone操作



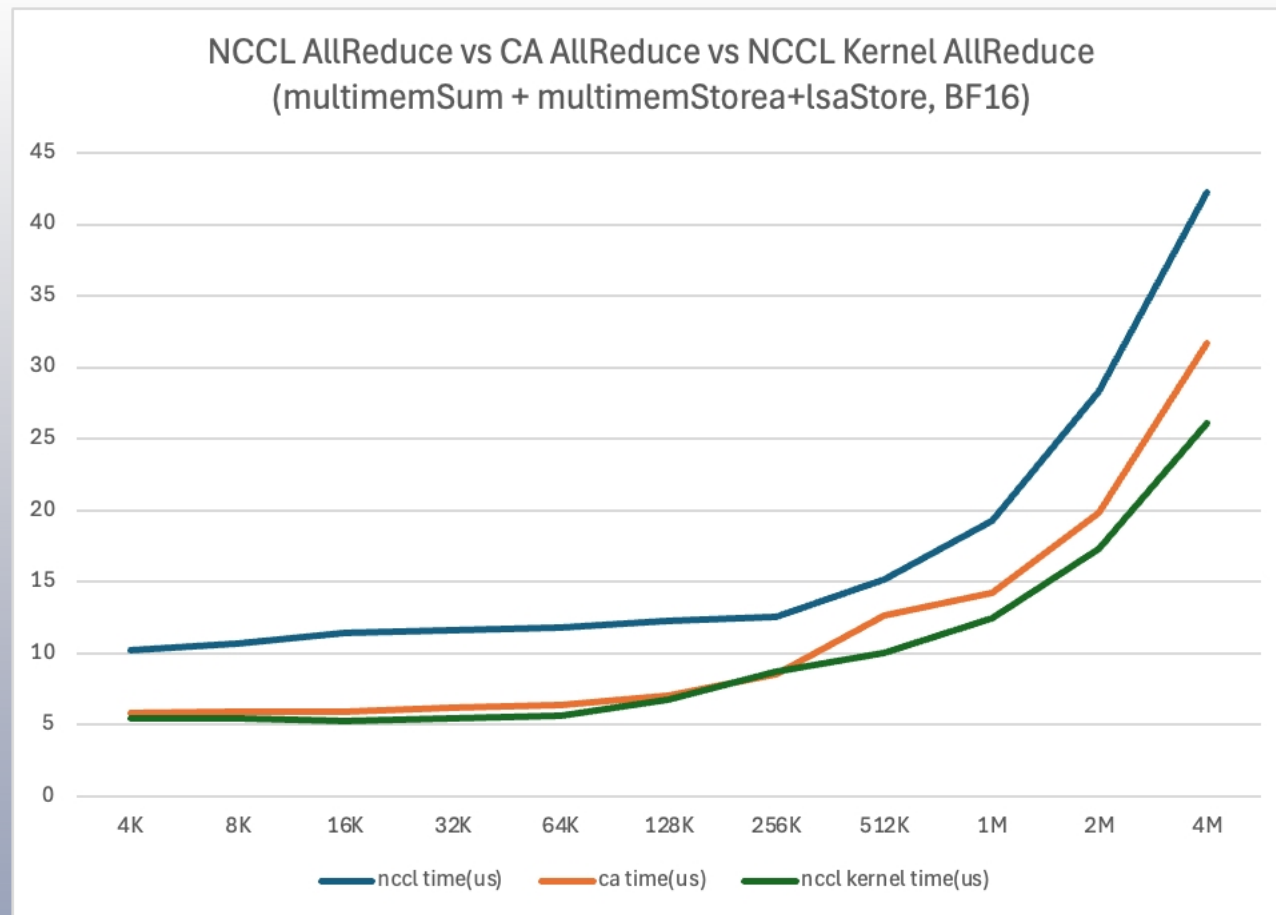
➤ 耗时占比大的FlagGems算子优化

- 向量化读取数据
- 使用TLE的shared memory功能加速计算



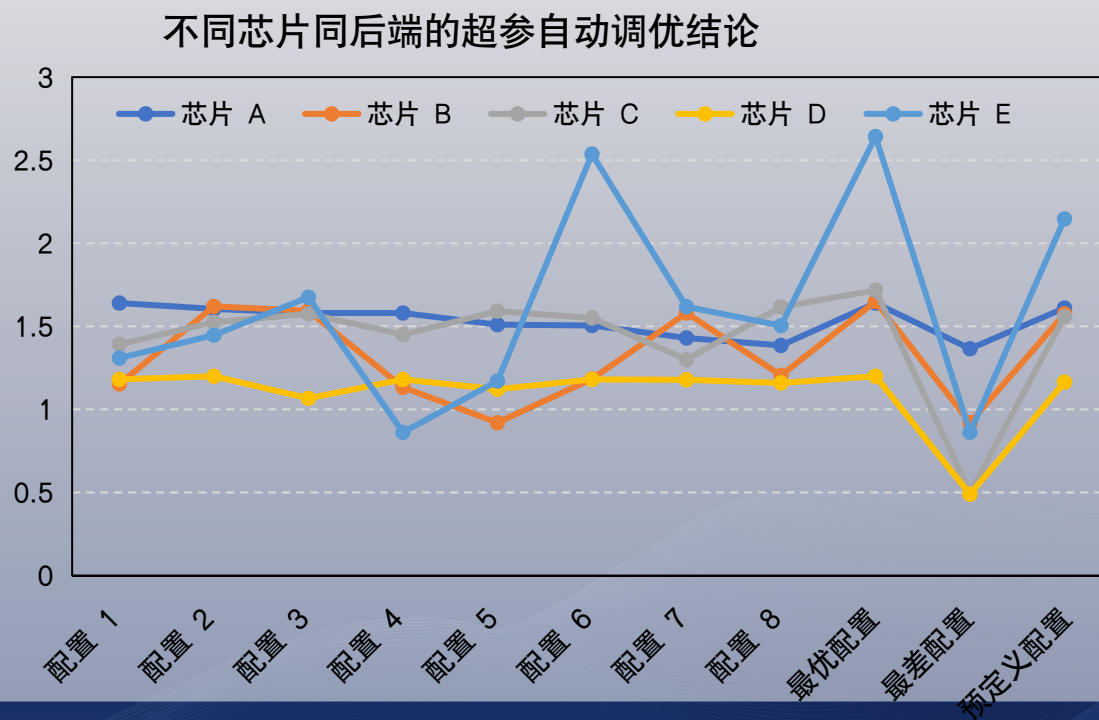
➤ FlagCX-CA通信算子优化

- FlagCX ncclAdator支持基于NCCL device API的 customAllReduce, 支持LSA/Multimem原语。
- 小消息 (<=512K) localAllReduce (MultimemSum + LSASore) 对比vLLM cross_device_1stage (IPC-based packed_reduce) 性能持平。
- 中小消息 ((512K, 8M]) interleavedAllReduce (MultimemSum + MultimemStore) 对比vLLM cross_device_2stage(IPC-based reducescatter + allgather), 性能小幅提升。



- **CUDA Platform:** 根据获取到的GPU架构型号进行名称匹配。对英伟达设备架构进行识别，针对性的对超参和fused_moe_kernel算子的自动调优配置进行匹配。
- **FlagOS Platform:** 多芯片版本，暂时无法针对某款芯片做架构名称识别。但是仍然需要兼顾性能和兼容性。

- 短期解决方案：
 - 对特化的优化进行手动设置。
- 长期解决方案：
 - 对设备架构等信息进行详细设计。引入超参或者配置的自动调优。



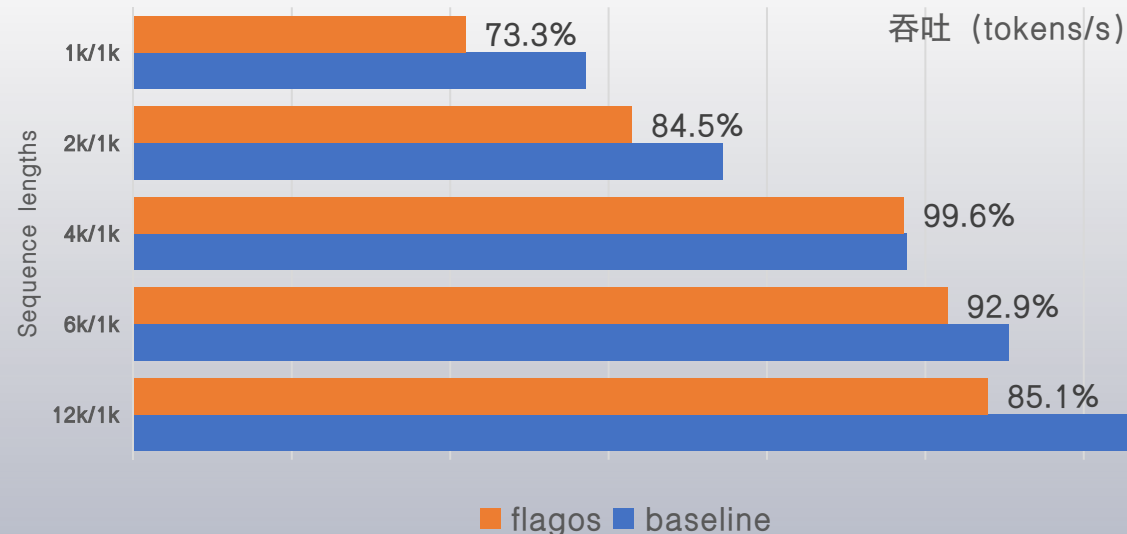
Qwen3-Next triton optimization

FlagOS

fused_moe_kernel
nvjet_tst_64x8_64x16_4x2_v_bz_TNT
void vllm::cross_device_reduce_1stage<__nv_b
void vllm::moe::topkGatingSoftmax<(int)16, (in
fused_recurrent_gated_delta_rule_fwd_kernel
nvjet_tst_64x8_64x16_1x1_v_bz_TNT
void at::native::reduce_kernel<(int)128, (int)4, a
void at::native::elementwise_kernel<(int)128, (i
void at::native::elementwise_kernel<(int)128, (i
void cutlass::device_kernel<flash::enable_sm9
void vllm::moe::moe_align_block_size_kernel<i
nvjet_tst_64x8_64x16_1x2_h_bz_TNT
void at::native::vectorized_elementwise_kernel
void at::native::elementwise_kernel<(int)128, (i
void vllm::moe::count_and_sort_expert_tokens
void vllm::act_and_mul_kernel<c10::BFloat16, &
_causal_conv1d_update_kernel

算子替换

fused_moe_kernel
moe_align_block_size_stage3
mm_kernel_general
void vllm::cross_device_reduce_1stage<__nv_bfloat16, (int)4
mm_kernel_general_host_tma
moe_align_block_size_stage2
fused_recurrent_gated_delta_rule_fwd_kernel
topk_gating_softmax_kernel
kernel_unified_attention_3d
_copy_kernel_cppwrapper_rank_4
mul_func_kernel_rank_2
zeros_kernel
moe_sum_kernel
fill_scalar_kernel
moe_align_block_size_stage4
moe_align_block_size_stage1
sigmoid_forward_kernel_rank_1
sum_dim_kernel_inner
_causal_conv1d_update_kernel
true_div_func_kernel_rank_2
silu_and_mul_kernel_kernel_rank_2
silu_forward_kernel_rank_2
_copy_kernel_cppwrapper_rank_2
fused_gdn_gating_kernel



- ✓ 实现Qwen3、Qwen3-Next 在单卡/多卡基于FlagOS 统一后端的端到端推理，达到100% triton算子替代。
- ✓ Qwen3-Next模型吞吐在英伟达上最优可达到原生的99.6%。

```
1  vllm_fl/
2  |  |__ __init__.py      # 插件入口点注册
3  |  |__ platform.py     # 自定义平台类 PlatformFL
4  |  |__ utils.py        # 工具函数: 设备检测、FlagGems 配置
5  |  |__ envs.py         # 环境变量处理
6  |
7  |  |__ worker/
8  |  |  |__ worker.py     # worker 抽象或调度逻辑, 例如处理请求、与主进程通信。
9  |  |  |__ model_runner.py # 对具体模型推理过程的封装, 例如加载模型、执行前向、处理批次等。
10 |
11 |  |__ compilation/
12 |  |  |__ graph.py      # 图相关的逻辑, 如果硬件支持图模式, 在这里适配。
13 |  |  |__ distributed/  # 通信相关的, 使用FlagCX作为通信库
14 |  |  |__ dispatch/    # 核心: 算子分发机制
15 |  |  |  |__ manager.py  # OpManager: 分发管理器
16 |  |  |  |__ registry.py # OpRegistry: 算子注册表
17 |  |  |  |__ types.py    # 核心类型: OpImpl, BackendImplKind
18 |  |  |  |__ policy.py   # SelectionPolicy: 选择策略
19 |  |  |  |__ builtin_ops.py # 内置算子注册
20 |  |  |  |__ discovery.py # 插件发现机制
21 |  |
22 |  |  |__ config/       # 平台特定配置
23 |  |  |  |__ ascend.yaml # Ascend NPU 默认配置
24 |  |  |  |__ cuda.yaml  # NVIDIA CUDA 默认配置
25 |  |
26 |  |  |__ backends/    # 后端实现
27 |  |  |  |__ flaggems/  # FlagGems 后端 (优先级 150)
28 |  |  |  |__ reference/ # PyTorch 参考实现 (优先级 50)
29 |  |  |  |__ vendor/    # 厂商后端 (优先级 100)
30 |  |  |  |  |__ cuda/    # NVIDIA CUDA 后端
31 |  |  |  |  |__ ascend/  # 华为 Ascend NPU 后端
32 |  |  |  |  |__ template/ # 新厂商后端模板
33 |
34 |  |__ ops/            # OOT (Out-of-Tree) 算子, 主要为一些融合算子
35 |  |__ models/        # 模型特定实现
```

➤ 统一的执行入口和执行流程。

➤ 算子层面厂商适配:

✓ Torch Aten算子替换:

- FlagOS: torch aten算子统一替换为FlagOS实现版本, 实现跨平台零成本迁移。

✓ vLLM融合算子dispatch:

- FlagOS 推荐后端: FlagOS 后端, 当前仅注册实现了部分基于CustomOP管理的 Triton OP。
- vendor 厂商后端: 当前包括原生vLLM CustomOP后端, 不同厂商的优化算子。

➤ 一次注册，多芯片适用。



Supported Chips

Chip Vendor	Status	Reference
NVIDIA	Supported	-
Ascend	Supported	-
MetaX	Merging	PR #90
Pingtouge-Zhenwu	Supported	-
Iluvatar	Supported	-
Tsingmicro	Merging	PR #52
Moore Threads	Merging	PR #97
Hygon	Supporting	-

- vLLM-FlagOS多芯片版本的迁移:
- 在不同芯片上都进行了文本和多模态数据的精度评测。
 - 保证发布的每个模型精度都有保障。

数据集	官方	vLLM(nvidia)	vLLM-FlagOS(芯片A)	vLLM-FlagOS(芯片B)
ERQA	67.5	65.28	67.79（采样150条）	67.33（采样150条）
AIME	91.3（AIME2026）	90	93.33	93.33

➤ model-migrate-fl

一条命令完成新模型从 vllm upstream 到 Plugin 的全自动迁移

➤ 核心能力

- 自动提取upstream vLLM最新模型代码
- 智能适配vLLM 0.13.0 API差异，自动打补丁
- 自动生成Config Bridge，解决跨版本配置兼容
- 内置13步标准化流程，中断可恢复
- 端到端Token级精度验证 (vs upstream GT)

➤ 全自动闭环验证

- One-Command: /model-migrate-fl <model> 全流程自动化
- 跨机编排验证: SSH → Docker → Conda 全链路自动化
- 确定性对比: temperature=0 + seed 保证可复现性
- 迭代闭环: 精度不对齐 → 自动定位 → 修复 → 重验
- AI Agent驱动: 理解代码语义，非简单文本替换

➤ 实践模型

- Kimi-k2.5, GLM-5, Qwen3.5

标准化迁移流程 (13步 / 5阶段)

准备阶段 (Step 1-2)

基线测试 → Clone upstream 源码

分析调研 (Step 3-4)

解析模型架构 → 学习Plugin已有模式

代码迁移 (Step 5-7)

Config Bridge → Copy-Patch模型 → 注册

质量验证 (Step 8-10)

代码审查 → 回归测试 → 功能测试

端到端验证 (Step 11-13)

Benchmark → Serve请求 → E2E精度对齐(vs GT)

Qwen3.5-397B 验证: Text 5/5 PASS

➤ 机制优化:

- 量化推理模块逻辑进行设计优化、统一管理。
- 对FusedMoE模块机制进行优化。

➤ 性能优化:

- 多芯片共性层面的性能优化，例如调度层面。
- 多芯片结合算子库等特性层面的性能优化。
- 利用PyTorch的图层Pass优化机制实现不同芯片的自定义融合等优化。
- 结合统一通信库FlagCX进行Prefill-Decode异构算力性能优化。

➤ 多芯片量化工具:

- 不同厂商使用不同的量化模型，保障精度性能



wechat



github repo



feishu group

Thanks

